Catalogue-Check — HRD-018 Foundation

Catalogue-Check — HRD-018 Foundation

Field	Value
Revision	1
Created	2026-05-20
Last modified	2026-05-20
Status	active
Status summary	Catalogue-Check survey executed per Universal §11.4.74 before Foundation M1 code lands. 8 capabilities probed against `vasic-digital` + `HelixDevelopment` orgs. **9 of 12 capabilities resolve to `extend` against existing Helix-stack modules** — major reuse opportunity.
Issues	HRD-018
Issues summary	see `../Issues.md`
Fixed	—
Fixed summary	—
Continuation	M1 implementation proceeds using the modules listed below. Each new go.mod dependency MUST cite this file.

Table of contents

§0. Purpose

Per Universal §11.4.74 (submodule-catalogue-first discovery), every new HRD MUST record a `Catalogue-Check: reuse | extend | no-match` verdict per capability *before* Go code lands. This file is the evidence record for HRD-018 (Foundation / `commons_constitution` scaffold + Postgres + REST).

§1. Method

Surveyed via `gh repo list` + `gh repo view` + `gh api repos/<org>/<name>/contents/`:

- `gh repo list vasic-digital --limit 200`
- `gh repo list HelixDevelopment --limit 200`
- For each shortlisted candidate, fetched `go.mod` + `README.md` + top-level file listing.

Probed 13 of ~125 listed repos in vasic-digital and 5 of 20 in HelixDevelopment. Survey ran 2026-05-20 between 15:50 and 16:10 local time. Repos surveyed (vasic-digital): `EventBus`, `database`, `middleware`, `auth`, `observability`, `recovery`, `BackgroundTasks`, `config`, `Messaging`, `Storage`, `cache`, `concurrency`, `security`. (HelixDevelopment): `HelixConstitution`, `HelixAgent`, `HelixMemory`, `LLMProvider`, `HelixSpecifier`.

§2. Capability-by-capability findings

§2.1 Constitution-rule Evaluator framework — no-match

No module in either org exposes a typed `Evaluator` interface with `RuleID`/`Severity`/`Evaluate`. Constitution-rule semantics are bespoke to Helix governance.

Verdict: Write new code in `commons_constitution/evaluator.go`.

§2.2 CloudEvents v1.0 emission — extend (`digital.vasic.eventbus`)

`digital.vasic.eventbus` (1.0+, Go 1.25, last commit 2026-05-20) provides typed events, dot-notation topics, glob/prefix/metadata filtering, middleware chain (Logging/Metrics/Enrich/RateLimit), Publish/PublishAsync/Wait/WaitMultiple, race-tested concurrent pub/sub, bounded subscriber buffers. Native event type is `digital.vasic.eventbus/pkg/event.Event` — not CNCF CloudEvents v1.0, but the abstractions are isomorphic.

Verdict: Depend on `digital.vasic.eventbus` for in-process pub/sub. Build a thin **`commons_constitution/cloudevents.go`** adapter that translates between `pkg/event.Event` and `cloudevents.Event` from `github.com/cloudevents/sdk-go/v2` for ingest/egress at the HTTP boundary only.

Repo URL: `https://github.com/vasic-digital/EventBus`

§2.3 Bundle-hash captureer — no-match

`digital.vasic.Document` does change tracking but at the document-model level (format detection), not a deterministic SHA-256 of a rendered Markdown bundle.

Verdict: Write new code in `commons_constitution/bundle.go` — trivial (~30 LOC: read file, SHA-256, hex).

§2.4 Mode-ladder — no-match

No module covers per-tenant per-rule allow/warn/enforce ladder.

Verdict: Write new code in `commons_constitution/ladder/*`.

§2.5 golang-migrate + embed FS — extend (`digital.vasic.database/pkg/migration`)

`digital.vasic.database` (Go 1.25, `github.com/jackc/pgx/v5 v5.9.2`, `gorm.io/gorm v1.31.1`) ships `pkg/migration` (version-tracked Up/Down with `schema_migrations` tracking table), `pkg/postgres` (pgxpool), `pkg/sqlite` (modernc — CGO-free), `pkg/pool` (generic `Pool[T]`), `pkg/repository` (generic `Repository[T]`), `pkg/query` (fluent SQL builder), `pkg/connection` (placeholder rewriting).

Verdict: Depend on `digital.vasic.database`. Pass our `//go:embed migrations/*.sql` `embed.FS` to `pkg/migration` via its existing source-driver hook. Remove the planned `commons_storage/migrate.go` and `commons_storage/pgx.go` files — they collapse into thin re-exports of `digital.vasic.database/pkg/migration` + `pkg/postgres`.

Repo URL: `https://github.com/vasic-digital/database`

§2.6 Postgres job queue — EXTEND-REPLACE (`digital.vasic.background`)

`digital.vasic.background` (module path `digital.vasic.background`, Go 1.25.3) is a Postgres-backed persistent task queue with:

- Priority-aware enqueue/dequeue
- Dynamic worker pool with resource-aware (CPU/memory) allocation
- Stuck detection via heuristics
- Event publishing for task lifecycle
- Pause/Resume via checkpointing
- Dead Letter Queue
- Progress reporting + heartbeat monitoring
- Prometheus metrics built-in (`github.com/prometheus/client_golang`)

This is functionally equivalent to River (`github.com/riverqueue/river`) but is a Helix-stack module under our control.

Verdict: Replace the planned River dependency with `digital.vasic.background`. This is the single biggest architectural change driven by the catalogue-check. Foundation no longer takes a dependency on `github.com/riverqueue/river`; instead `commons_storage` adapts `digital.vasic.background.WorkerPool` for audit-fanout and channel-dispatch jobs.

Repo URL: `https://github.com/vasic-digital/BackgroundTasks`

§2.7 pgx + RLS tenant-context middleware — extend (`digital.vasic.database/pkg/postgres`)

`digital.vasic.database/pkg/postgres` provides the pgxpool wrapper. RLS `SET LOCAL app.tenant_id = ...` is a thin call we add via a small wrapper.

Verdict: Add `commons_storage/tenant_context.go` (~40 LOC) that takes a `digital.vasic.database/pkg/postgres.DB` + a tenant UUID and runs the RLS GUC inside a transaction lifetime. No pgx pool wrangling on our side.

§2.8 Gin + JWT + OTel + recovery + requestid — STRONG-EXTEND (3 modules)

- **`digital.vasic.middleware`** (Go 1.25, `github.com/gin-gonic/gin v1.12.0`) — packages: `chain`, `cors`, `logging`, `recovery`, `requestid`, `auth`, `validation`, `ratelimit`, `brotli`, `cache`, `altsvc`, `gin` (Gin adapter), `i18n`. All middleware honors `func(http.Handler) http.Handler` for framework portability; Gin adapter is a thin bridge.
- **`digital.vasic.auth`** (Go 1.25, `github.com/golang-jwt/jwt/v5 v5.2.2`) — packages: `pkg/jwt` (Manager + Create/Validate/Refresh), `pkg/apikey` (Generator + Store + masking + scopes), `pkg/oauth` (file-reader + HTTP refresher + auto-refresh), `pkg/middleware` (Bearer + API-key + scope-validation middleware), `pkg/token` (Token + Claims + in-memory store with TTL/revoke), `pkg/i18n`.
- **`digital.vasic.observability`** — OTel module (probed only at top-level; deep contents will be probed at M3 wiring time).

Verdict: Foundation's `pherald/internal/http/middleware.go` becomes a 30-line composition of `digital.vasic.middleware.chain.Chain(...)` with `requestid.Middleware()`, `digital.vasic.auth.middleware.Bearer()`, `digital.vasic.middleware.recovery.Recovery()`, and `digital.vasic.observability.OTel()`. The JWT manager is `digital.vasic.auth.jwt.NewManager()`.

Repo URLs:

- `https://github.com/vasic-digital/middleware`
- `https://github.com/vasic-digital/auth`
- `https://github.com/vasic-digital/observability`

§2.9 Redis cache wrapper — extend (`digital.vasic.cache`)

Repo present with full Helix structure (CLAUDE/AGENTS/Constitution/ ARCHITECTURE docs + Makefile + go.mod + go.sum). Deep content not yet probed.

Verdict: Depend on `digital.vasic.cache` for the M3 read-cache wrapper around the mode-ladder. Deep probe at M3 wiring time to confirm Redis-backend availability.

Repo URL: `https://github.com/vasic-digital/cache`

§2.10 Config loading — extend (`digital.vasic.config`)

Use for §6 config-block loading. Deep probe deferred to wiring time.

Repo URL: `https://github.com/vasic-digital/config`

§2.11 Panic recovery — extend (`digital.vasic.recovery`)

Use for `Registry.runOne` panic isolation per §4 row 1 of the design.

Repo URL: `https://github.com/vasic-digital/recovery`

§2.12 In-process pub/sub — extend (`digital.vasic.eventbus`)

Replaces the planned Watermill memory-pubsub. Same module as §2.2.

§3. Foundation dependency manifest

After the pivot, Foundation's `go.mod` consumes these new modules (paths use the `digital.vasic.*` namespace per the constitutional module-naming convention surfaced in the catalogue):

```
require (
    digital.vasic.eventbus      v0.0.0    // §2.2 + §2.12 — pub/
        sub + CloudEvents emission base
    digital.vasic.database      v0.0.0    // §2.5 + §2.7 — pgx
        pool + migrations + RLS host
    digital.vasic.background    v0.0.0    // §2.6 — Postgres job
        queue (replaces River)
    digital.vasic.middleware    v0.0.0    // §2.8 — Gin + chain +
        recovery + requestid + ratelimit
    digital.vasic.auth          v0.0.0    // §2.8 — JWT + Bearer
        middleware
    digital.vasic.observability v0.0.0    // §2.8 — OTel
    digital.vasic.cache         v0.0.0    // §2.9 — Redis cache
    digital.vasic.config        v0.0.0    // §2.10 — config
        loading
```

```
    digital.vasic.recovery      v0.0.0      // §2.11 — panic
        recovery
)
```

Each module enters the workspace as a **git submodule under `submodules/<name>/`** (per `vendored SDKs` policy in CLAUDE.md: "any official/unofficial messenger SDK or API client we depend on goes in as a **git submodule** … — not `go get`'d into `go.mod`"). The `go.work` file references each submodule path; `go.mod replace` directives point at the local paths during development. Production builds resolve through pinned commit SHAs.

§4. Architectural pivot summary

The user-approved design (`docs/superpowers/specs/2026-05-20-foundation-design.md` §1–§5) referenced `Watermill` and `River` as external standards. Post-catalogue-check, these references shift:

Original design ref	Catalogue verdict	New ref
`github.com/ThreeDotsLabs/watermill` memory-pubsub	extend	`digital.vasic.eventbus`
`github.com/riverqueue/river` job queue	EXTEND-REPLACE	`digital.vasic.background`
Raw `github.com/gin-gonic/gin` server skeleton	strong-extend	`digital.vasic.middleware` + `digital.vasic.auth` composition
Raw `github.com/jackc/pgx/v5/pgxpool` pool + golang-migrate	extend	`digital.vasic.database`
Raw `github.com/redis/go-redis/v9` client	extend	`digital.vasic.cache`
Custom OTel wiring	extend	`digital.vasic.observability`

The smoke-test contracts (M1/M2/M3) and the data-flow trace (§3.1 of the design) are unchanged — only the libraries that implement them shift. The bespoke `commons_constitution` layer (Evaluator + 12 emit helpers + BundleHash + ModeLadder + ConstitutionStore) remains entirely new code.

The transition-gate, three-axis envelope, mode-ladder semantics, RLS isolation contract, and per-milestone smoke definitions all hold.

§5. Reproducibility

To re-run this catalogue-check:

```
gh repo list vasic-digital --limit 200 --json
        name,description,updatedAt > /tmp/vasic-digital.json
gh repo list HelixDevelopment --limit 200 --json
        name,description,updatedAt > /tmp/helix-development.json

for repo in EventBus database middleware auth observability
        recovery BackgroundTasks config Messaging Storage cache
        concurrency security; do
    gh api repos/vasic-digital/$repo/contents/go.mod > /tmp/cc-
        $repo.gomod.json
    gh api repos/vasic-digital/$repo/contents/README.md > /tmp/cc-
        $repo.readme.json
done
```

Compare timestamps in this file's frontmatter against the `updatedAt` values to confirm survey is fresh.